

dbt Interview Questions - Incremental & Snapshots

Focus: Incremental Models and Snapshots

dbt Interview Questions and Answers

Focus areas: Incremental models, Snapshot (SCD Type 2) logic, plus core dbt concepts likely asked alongside them

1. dbt Fundamentals

Q1: What is dbt, and where does it sit in a modern data stack?

dbt (data build tool) handles the **T in ELT** - it does not extract or load data, it only transforms data that is already sitting in your warehouse (Snowflake, BigQuery, etc.). It compiles SQL + Jinja templates into plain SQL and runs it against your warehouse connection, adding version control, testing, documentation, and dependency management (DAG) on top of plain SQL scripts.

Q2: What are the dbt materializations, and when do you use each?

- **view:** query saved as a view, no data stored - fast to build, slower to query, used for lightweight staging models. - **table:** full rebuild every run - good for smaller/stable dimension tables or marts that need fast read performance. - **incremental:** only processes new/changed rows on each run instead of a full rebuild - used for large fact tables where a full rebuild is too slow/expensive. - **ephemeral:** not built as a database object at all; inlined as a CTE into downstream models - used for lightweight reusable logic that doesn't need its own object.

Q3: What's the difference between a dbt source, a seed, and a model?

- **source:** a reference to raw data already loaded into your warehouse by an external process (e.g., your ADF/Snowflake framework loading SAP data into Bronze). - **seed:** a static CSV file checked into your dbt project and loaded as a table (e.g., a small reference/lookup file - country codes, classification mappings). - **model:** a .sql file containing a SELECT statement that dbt turns into a table/view.

2. Incremental Models

Q1: What is an incremental model, and why use one?

An incremental model only inserts/updates the **new or changed rows** since the last run, instead of rebuilding the entire table every time. This matters for large fact tables (e.g., SAP Sales/Procurement data at BHP scale) where a full rebuild would be slow and expensive to re-scan and re-write every run.

Q2: Write a basic incremental model.

```
{{
  config(
    materialized='incremental',
```

```

        unique_key='order_id'
    )
}}

SELECT
    order_id,
    customer_id,
    order_amount,
    load_timestamp
FROM {{ source('bronze', 'raw_orders') }}

{% if is_incremental() %}
    WHERE load_timestamp > (SELECT MAX(load_timestamp) FROM {{ this
}})

{% endif %}

```

Q3: Explain `is_incremental()`. What does it actually check, and why does the filter only apply on some runs? `is_incremental()` returns true only when **all** of these are satisfied: the model already exists as a table in the warehouse, it's not a full-refresh run, and it's materialized as incremental. On the very first run (table doesn't exist yet), it's false, so dbt runs the query with no filter and builds the full table. On every subsequent normal run, it's true, so only new rows past the watermark get processed – this is the dbt-native equivalent of the watermark-table pattern in your ADF pipelines (LastLoadValue comparison), just expressed as `SELECT MAX(load_timestamp) FROM {{ this }}` instead of a separate control table.

Q4: What is `{{ this }}`? A reference to the model's **own** existing table in the database – used specifically to check “what's already loaded” during an incremental run (as in Q2/Q3 above).

Q5: What incremental strategies does dbt support, and how do you choose between them? - **append**: just inserts new rows, no update/dedup logic – fastest, but risks duplicates if the same row lands twice. - **merge**: (Snowflake, BigQuery, Databricks) does an actual MERGE – updates matched rows on `unique_key`, inserts unmatched ones. Most commonly used for slowly-changing fact data. - **delete+insert**: deletes rows matching the incremental filter, then inserts the new batch – useful when a whole day's/batch's data needs to be replaced rather than row-by-row merged. - **insert_overwrite**: (BigQuery, Spark) replaces entire partitions rather than individual rows – efficient when your source data is naturally partitioned by date.

Set it explicitly in config:

```

{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge'
)}}

```

Q6: What does `unique_key` actually do, and what happens if you don't set it? It tells dbt which column(s) identify a unique row so merge/delete+insert know what to match on for updates. Without it, dbt has no way to detect “this row already exists, update it” – it'll just append, which can create duplicates on re-runs or late-arriving/corrected records.

Q7: How would you handle a late-arriving or corrected record in an incremental model – a row that changed after your watermark already advanced past it? This is a common gotcha: a pure `load_timestamp > MAX(load_timestamp)` filter can permanently miss a row that arrives late with an *older* business date but a *newer* `load_timestamp`. A safer pattern uses a lookback window**:**

```

{% if is_incremental() %}
    WHERE load_timestamp > (SELECT MAX(load_timestamp) FROM {{ this
}}) - INTERVAL '3 days'

```

```
{% endif %}
```

combined with merge on unique_key so reprocessed rows update in place rather than duplicate.

Q8: How do you force a full rebuild of an incremental model?

```
dbt run --full-refresh --select my_incremental_model
```

This ignores the incremental logic for that run and rebuilds the table from scratch – necessary after a schema change, a backfill, or fixing a logic bug that already corrupted historical rows.

Q9: Your resume/project uses a custom framework with its own MERGE-based delta load (SAP -> Bronze). Where does dbt's incremental model fit relative to that? Good answer: the framework's merge handles **ingestion-level** delta loads (Bronze layer – raw SAP data landing). dbt's incremental models handle **transformation-level** delta processing further downstream (Silver->Gold/Conformed) – e.g., only reprocessing new/changed Bronze rows when building a Gold fact table, rather than re-transforming the entire history every run. Same underlying idea (don't reprocess unchanged data), applied at two different layers.

Q10: What's a common mistake people make with incremental models? Forgetting that **schema changes** (new column added to source) can silently break incremental models, because the existing table won't have the new column and a naive INSERT/MERGE can fail or silently drop it. dbt has an on_schema_change config (ignore, fail, append_new_columns, sync_all_columns) specifically to control this behavior.

3. Snapshots (SCD Type 2)

Q1: What is a dbt snapshot, and what problem does it solve? A snapshot captures **row-level changes over time** in a mutable source table, implementing **Slowly Changing Dimension Type 2** automatically. Without it, if a source row gets updated or deleted, you lose the ability to see what it looked like historically – a snapshot preserves that history by inserting a new versioned row every time it detects a change, rather than overwriting.

Q2: Where do snapshots live in a dbt project, and what does the config look like?

```
-- snapshots/orders_snapshot.sql
{% snapshot orders_snapshot %}

{{
    config(
        target_schema='snapshots',
        unique_key='order_id',
        strategy='timestamp',
        updated_at='updated_at_column'
    )
}}

SELECT * FROM {{ source('bronze', 'raw_orders') }}
```

Run with:

```
dbt snapshot
```

Q3: What's the difference between the timestamp strategy and the check strategy? - **timestamp**: dbt compares an `updated_at` column against the last snapshot record - if it's newer, a new version row is inserted. Requires a reliable, always-updated timestamp column on the source. - **check**: dbt compares specific columns (or all columns) row-by-row for **any** value change - used when the source has **no reliable updated_at column** (very common with SAP extracts, which is likely closer to your project's reality).

```
{{
  config(
    target_schema='snapshots',
    unique_key='order_id',
    strategy='check',
    check_cols=['status', 'amount', 'plant_name']
  )
}}
```

Or `check_cols='all_columns'` to compare every column.

Q4: What columns does dbt automatically add to a snapshot table, and what do they mean? - `dbt_scd_id`: a unique surrogate key for that specific version of the row. - `dbt_updated_at`: when dbt detected the change. - `dbt_valid_from`: when this version became active. - `dbt_valid_to`: when this version stopped being active - NULL means it's the **current** version. This is the same SCD Type 2 pattern as the `effective_start_date / effective_end_date / is_current` columns you'd have built manually at Mouritech - dbt just automates the detection and versioning logic instead of you hand-coding the compare-and-insert logic.

Q5: How would you query only the *current* version of a snapshot record?

```
SELECT * FROM {{ ref('orders_snapshot') }}
WHERE dbt_valid_to IS NULL;
```

Q6: What happens if a row is *deleted* from the source - does the snapshot know? By default, no - a hard delete at source is invisible to a standard snapshot (the row just stops appearing in the source query, but nothing in the snapshot marks it as ended). dbt added an `invalidate_hard_deletes=True` config specifically to handle this: when enabled, dbt checks for source PKs missing from the latest snapshot query and sets their `dbt_valid_to` to the current run's snapshot timestamp - effectively closing out that version. Worth mentioning proactively if asked "what are the limitations of snapshots."

Q7: Snapshot vs. incremental model - when would you use which? - **Incremental model**: you want the transformed table to always reflect the **current** state efficiently (no full rebuild) - you don't need to keep old versions of a row. - **Snapshot**: you specifically need **historical versions** of a mutable row for point-in-time analysis (e.g., "what was this SAP procurement order's status on March 1st") - this is about *preserving history*, not just processing efficiently. They solve different problems and are often used together: snapshot the raw mutable source, then build incremental models on top of the snapshot for the marts.

Q8: Since SAP extracts in your project are typically full/append file drops (not row-level updates), how would you decide whether a snapshot makes sense at all? Good critical-thinking answer: if Datavard/SAP consistently sends full extracts (not true CDC), a snapshot on the raw Bronze table can still work - dbt just compares the latest full extract against the last snapshot state and detects what changed, regardless of *how* the source delivered the data. The check strategy is well suited here since SAP data commonly lacks a trustworthy `updated_at` field.

Q9: Can you snapshot a source that's already an incremental dbt model? Not typically recommended – snapshots are meant to sit close to a raw/mutable source and capture its native change history. Snapshotting a model that's already been transformed/deduplicated can hide the actual source-level changes you're trying to preserve. Best practice: snapshot as early as possible (near Bronze/raw), then build incremental/table models downstream from the snapshot.

4. Testing, Sources, and Project Structure

Q1: What are the built-in generic tests in dbt, and when do you use each? - unique – no duplicate values (e.g., a primary key column). - not_null – column must never be null (e.g., required business keys). - accepted_values – column must only contain a defined set of values (e.g., data_classification IN ('Internal', 'Confidential', 'Highly Confidential')). - relationships – referential integrity check between a fact table's foreign key and a dimension table's primary key.

Q2: How do you write a custom (singular) test? A .sql file in the tests/ folder that returns rows **only when the test fails** – dbt treats any returned row as a failure.

```
-- tests/assert_positive_total_price.sql
SELECT *
FROM {{ ref('fct_sales') }}
WHERE total_price < 0
```

Q3: What's a dbt exposure, and why does it matter in a governed project like yours? An exposure declares a downstream consumer (e.g., a Power BI dashboard) of a dbt model in .yaml, so dbt docs and lineage graphs show the full path from raw source through to the actual dashboard – useful for impact analysis (“if I change this model, which dashboards break”) and for governance sign-off trails.

Q4: What does dbt build do differently from dbt run? dbt run only builds models. dbt build runs models, **and** tests, **and** snapshots, **and** seeds in the correct dependency order in a single command – closer to what you'd actually run in a CI/CD job.

Q5: How do you handle environment promotion (Dev -> QA -> Prod) in dbt – tying into your CR/CI-CD process? Via profiles.yaml targets (dev, qa, prod) pointing to different Snowflake databases/schemas, combined with your GitLab CI/CD pipeline running dbt build --target qa or --target prod at the appropriate promotion stage – consistent with your project's Dev->QA->Prod gate structure (CR, UAT sign-off) sitting around the dbt run itself.

Quick Reference: Answering “Walk me through incremental + snapshot together”

A strong end-to-end answer for a senior-level question: *“In our project, SAP data lands in Bronze via the framework's own merge-based delta load. From there, if I need full history of a mutable business record – say a procurement order that changes status multiple times – I'd snapshot it near Bronze using the check strategy, since SAP data usually doesn't have a trustworthy updated_at column. Then, for the Gold-layer fact tables that just need current state processed efficiently without reprocessing all history every run, I'd build those as incremental models with a merge strategy on the natural key, filtering on load_timestamp past the last watermark –*

same underlying idea as the ADF watermark pattern, just implemented as dbt's `is_incremental()` macro instead of a separate control table."